bthome-writable — a briefing for the BTHome maintainers

A briefing, not a draft: facts and links to speak from. The submission is the owner's own words — the Open Home Foundation AI policy closes contributions believed to be agent-written (D-091).

The ask

Reserve one object ID: `0xFF`. Everything else reuses BTHome's own object table, encodings and AES-CCM. And a question rather than a request: the specification is BTHome's to take if the maintainers want it.

Why it is worth having

Gordon Williams (Espruino), its co-designer:

> BTHome is a really great tool for sensors, but it's really handy to have a way of controlling bluetooth devices in a low-power, secure way, and nothing in Home Assistant really does that at the moment. It adds that functionality for those that want it but in a way that doesn't complicate BTHome.

His list of what it unlocks: clocks — the time, alarms; wireless lights; thermostats — a target temperature; configuration on wireless sensors — a movement threshold, a polling interval; plant waterers; epaper displays.

What it does

A device lists, in its ordinary BTHome advertising, the object types it accepts writes for — object `0xFF`, **last in the service data**, so existing receivers skip it. A receiver connects briefly and writes **one BTHome object** to that entry's own GATT characteristic. The write response is the acknowledgement; a writable value is never advertised.

```
advertising   40 00 09 01 61 FF 01 1E     battery, and "entry 1 accepts 0x1E"
write         1E 01  ->  2FAA0001         one object, one characteristic
```

It answers requests already made here

- **bthome-ble #257** — *"BTHome should define an Object Id for supported events"*: our declaration, asked for independently in 2025 for a narrower case.
- **#146** two-way communication — Ernst79: *"we will welcome contributions from others if they want to add this somehow."* And **#287**, controlling Shelly BLU devices.

Re-checked 2026-10-13: no downlink proposed anywhere, `0xFF` unassigned.

Evidence

Three benches, two of them other people's. Gordon Williams on a Puck.js: *"works great"*, connections brief, no missed writes. @enaon on an RPi4b with OpenWrt, twenty encrypted lights, six ESPHome proxies and local BlueZ: working over both paths. **Both outside reports found faults this bench could not produce**, each fixed with a test that fails without the fix — which is an argument for reviewing this rather than for trusting it.

Measured through Home Assistant	
Click to action, 1 s advertising interval	**1.7 s**, of which 36 ms is the write
Repeated commands	**0.31 s**
Delivered, two devices	**100 %**, no retries
Declaration on the air, three writable lights	**5 bytes**

Two reference implementations, and **the device side ships in Espruino's own module library** (merged 2026-10-12). A shared test-vector contract, whose AES-CCM vectors three implementations agree on, and 716 automated tests. The installation guides have been walked end to end using nothing else: three documentation

defects, no protocol or code defect (D-093).

Objections, and the short answers

- **Slow, and costly in battery.** 1.7 s click to action, and the idle advertising interval is the device's own choice, unchanged by this.
- **Unknown objects break existing receivers.** `bthome-ble` skips an unknown ID and stops parsing, so the declaration goes last and existing installs lose nothing. Verified against the real parser in CI.
- **Writes are a security hole.** BTHome's own AES-CCM both directions, the direction bound into the nonce so no recording replays as another, and a monotonic counter. Without the key there is no write.

What adoption would mean, in steps

Each step is worth something on its own, and none of them forces the next.

1. **Reserve** `0xFF` for the declaration object. That is the whole of the hard dependency: with the ID assigned, this stops occupying an unassigned value and parsers can skip it knowingly rather than by accident.
2. **Take the specification**, if the maintainers want it. The object table, the encodings and the encryption are BTHome's throughout; what is new is one object and a GATT profile.
3. **Fold the receiver into core** `bthome`, eventually. Gordon Williams: *"nothing would stop it being built in to BTHome itself eventually."*

Both reference implementations stay maintained either way.

What it costs an installation that does not want it

Nothing measurable. The declaration is five bytes at the **end** of the service data — `bthome-ble` stops at the first object ID it does not know, which is exactly why the rule is that it goes last. A device that implements none of this is untouched, and so is a receiver that ignores `0xFF`.

The security model, briefly

BTHome's own AES-CCM in both directions, with the **direction bound into the nonce** — `0x41` advertising, `0xFF` write, `0xFE` read — so nothing captured in one direction authenticates in another. A write carries a monotonic counter the device checks *before* spending an AES on it. Advertising is filtered with `bthome-ble`'s own replay rule, borrowed rather than invented. Without the key there is no write, and a writable value is never advertised.

Known limitations, stated rather than waited for

- **A silently refused write still looks delivered.** The device acknowledges a write before validating it, because Espruino cannot yet fail one at the ATT layer. The causes are being removed one at a time; the class closes when the firmware can refuse.
- **A plaintext packet from a keyed device is accepted**, exactly as core `bthome` accepts one. The check belongs in `bthome-ble`, where every BTHome device would gain it, rather than bolted onto one extension (D-090).
- **UUIDs are provisional**, and the counter report — the one thing a receiver cannot work out for itself after a device restarts — is implemented on both sides but not yet agreed.

Links

https://github.com/yerpj/bthome-writable — specification `spec/PROTOCOL.md`, every resolved question with the measurement that resolved it `spec/decisions.md`, ten-minute replication `docs/try-it.md`. Discussion: espruino#8024.